LOW-LEVEL DESIGN (LLD)

RAG-Based Customer Support Assistant using LangGraph & HITL

1. Document Processing Module

Responsibility:

- Load PDF file
- Extract raw text

Implementation:

- Uses PyPDFLoader
- Input: PDF file (data/sample.pdf)
- Output: List of document objects

2. Chunking Module

Responsibility:

- Split large text into smaller chunks

Implementation:

- Uses RecursiveCharacterTextSplitter
- Parameters:
    - Chunk size: 500
    - Chunk overlap: 100

Output:

- List of text chunks

3. Embedding Module

Responsibility:

- Convert text chunks into vectors

Implementation:

- Uses HuggingFaceEmbeddings
- Model: all-MiniLM-L6-v2

Output:

- Vector embeddings for each chunk

4. Vector Storage Module

Responsibility:

- Store embeddings efficiently

Implementation:

- Uses ChromaDB
- Persist directory: db/

Operations:

- Insert embeddings
- Retrieve embeddings

5. Retrieval Module

Responsibility:

- Retrieve relevant chunks based on query

Implementation:

- Similarity search using ChromaDB
- Top-K retrieval (k=3)

Input:

- User query

Output:

- Relevant document chunks

6. Query Processing Module

Responsibility:

- Generate response using retrieved data

Implementation:

- Uses HuggingFace model (Flan-T5)

Steps:

- Combine retrieved chunks into context
- Create prompt
- Generate answer

7. Graph Execution Module

Responsibility:

- Manage workflow execution

Implementation:

- Simulates LangGraph flow
- Controls:
    - Input → Processing → Output

8. HITL Module

Responsibility:

- Handle human escalation

Implementation:

- Triggered when:
    - No relevant data
    - Low confidence answer

Output:

- Human-provided response

2.Data Structures

Document Representation

```
{
  "page_content": "text from PDF",
  "metadata": {
    "source": "page number"
  }
}
```

Chunk Format

```
{
  "chunk_id": "unique_id",
  "text": "chunk content"
}
```

Embedding Structure

```
{
  "vector": [0.12, 0.45, ...],
  "chunk_id": "reference_id"
}
```

Query-Response Schema

```
{
  "query": "user input",
  "context": "retrieved chunks",
  "response": "generated answer",
  "source": "LLM / HITL"
}
```

Graph State Object

```
{
  "query": "",
  "context": "",
  "answer": "",
  "confidence": ""
}
```

3. Workflow Design (LangGraph)

Nodes

1. **Processing Node**
    - Retrieves data
    - Generates answer
2. **Output Node**
    - Returns final response

Edges

- Input → Processing

- Processing → Output
- Processing → HITL (conditional)

State Flow

- Query → Context → Answer → Decision

4. Conditional Routing Logic

Answer Generation Criteria

- Relevant chunks found
- Context is sufficient

Escalation Criteria

System triggers HITL when:

- No chunks retrieved
- Answer contains “I don’t know”
- Answer length is too short
- Query is outside document scope

5. HITL Design

When Triggered

- Out-of-scope query
- Missing context

Process

1. System detects low confidence
2. Escalation message displayed
3. Human provides input
4. Response returned to user

Integration

- Human response replaces LLM output
- Maintains workflow continuity

5. API / Interface Design

Input Format

```
{
  "query": "What is sales management?"
}
```

Output Format

```
{
  "response": "Generated answer",
  "source": "LLM / HITL"
}
```

Interaction Flow

- User enters query via CLI
- System processes query
- Returns response

Error Handling

Missing Data

- If PDF not found → raise error

No Relevant Chunks

- Trigger HITL

LLM Failure

- Return fallback message
- Escalate to human

Embedding Failure

- Retry or log error

Conclusion

The Low-Level Design provides a detailed breakdown of how each component is implemented and interacts internally. It ensures modularity, scalability, and maintainability while enabling accurate retrieval and intelligent response generation with HITL support.